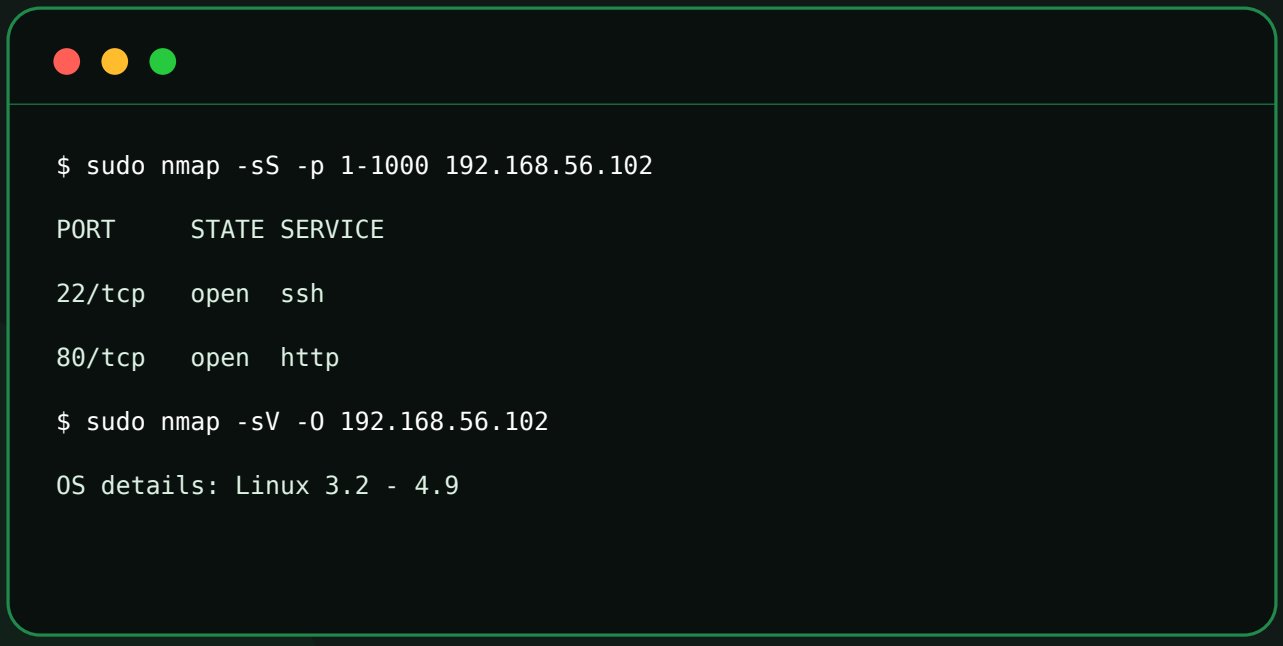
PRAXISHANDBUCH

Nmap

Das Praxishandbuch

Netzwerke systematisch erkunden – von der Host-Erkennung bis zur Scripting Engine



SYSTEMATISCH ERKUNDEN. JEDEN FUND VERSTEHEN. NUR IM EIGENEN NETZ.

Mit Übungen, Spickzettel, Glossar und Index · Ausgabe Oktober 2026

Nmap
Das Praxishandbuch

Netzwerke systematisch erkunden – von der Host-Erkennung bis zur Scripting Engine

Von Host-Discovery über Port- und Versionserkennung bis zur Nmap Scripting Engine (NSE). Mit geführten Übungen in einem eigenen, isolierten Testnetz, Spickzettel, Glossar und Index.

Inhaltsverzeichnis

Vorwort 5

Für wen ist dieses Handbuch? 5

Aufbau 6

Kapitel 1: Was ist Nmap? 7

1.1 Einordnung: automatisierte Erkundung statt manueller Paketbau 7

1.2 Typische Einsatzgebiete 7

1.3 Der grundsätzliche Scan-Ablauf 8

1.4 Dual-Use wie jedes Erkundungswerkzeug 8

Zusammenfassung 8

Kapitel 2: Host-Discovery 9

2.1 Warum ein eigener Schritt vor dem Portscan? 9

2.2 Discovery-Methoden im Überblick 9

2.3 Discovery überspringen oder erzwingen 10

Zusammenfassung 10

Kapitel 3: Installation und Laborumgebung 11

3.1 Installation 11

3.2 Root-Rechte: wann nötig, wann nicht 11

3.3 Das isolierte Testnetz 12

Zusammenfassung 12

Kapitel 4: Scan-Typen 13

4.1 Die wichtigsten TCP-Scan-Typen 13

4.2 UDP-Scans 13

4.3 Ports gezielt auswählen 14

Zusammenfassung 14

Kapitel 5: Service-, Versions- und Betriebssystemerkennung 15

5.1 Service- und Versionserkennung 15

5.2 Betriebssystem-Fingerprinting 15

5.3 Beides kombinieren 16

Zusammenfassung 16

Kapitel 6: Timing und Performance 17

6.1 Die Timing-Templates . . . 17
6.2 Feinere Kontrolle jenseits der Templates . . . 17
6.3 Der Zielkonflikt . . . 18
Zusammenfassung . . . 18

Kapitel 7: Die Nmap Scripting Engine (NSE) . . . 19

7.1 Was ist NSE? . . . 19
7.2 Skript-Kategorien . . . 19
7.3 Ein einzelnes Skript gezielt einsetzen . . . 20
7.4 Eigene Argumente an Skripte übergeben . . . 20
Zusammenfassung . . . 20

Kapitel 8: Firewall- und IDS-Evasion . . . 21

8.1 Fragmentierung . . . 21
8.2 Decoys: Scheinbar viele gleichzeitige Scanner . . . 21
8.3 Quellport-Manipulation . . . 22
8.4 Idle Scan mit Nmap . . . 22
8.5 Warum Evasion heute begrenzt wirkt . . . 22
Zusammenfassung . . . 22

Kapitel 9: Ausgabeformate und Automatisierung . . . 23

9.1 Die vier Ausgabeformate . . . 23
9.2 Ergebnisse mit Shell-Werkzeugen auswerten . . . 23
9.3 XML als Grundlage für weitere Werkzeuge . . . 23
Zusammenfassung . . . 24

Kapitel 10: Praxisteil – Laboraufbau und geführte Übungen . . . 25

10.1 Laboraufbau . . . 25
10.2 Checkliste: sichere Laborumgebung . . . 25
10.3 Übungen . . . 25
Zusammenfassung . . . 27

Kapitel 11: Troubleshooting, Spickzettel, Glossar und Ressourcen . . . 28

11.1 Häufige Probleme . . . 28
11.2 Spickzettel . . . 28
11.3 Glossar . . . 29
11.4 Weiterführende Ressourcen . . . 29
Zusammenfassung . . . 29

Verzeichnis der Befehlsbeispiele **31**

Index **32**

Vorwort

`Nmap` (Network Mapper) ist das am weitesten verbreitete Werkzeug zur Netzwerkerkundung überhaupt. Wo `hping3` dir die volle, manuelle Kontrolle über jedes einzelne Paketfeld gibt, nimmt dir Nmap die Detailarbeit ab und automatisiert sie zu robusten, wiederholbaren Abläufen: Hosterkennung, Portscans, Diensterkennung, Betriebssystem-Fingerprinting und mit der eingebauten Scripting Engine sogar komplette Schwachstellenprüfungen.

Dieses Handbuch ergänzt das Ethical-Hacking-Handbuch und das hping3-Handbuch derselben Reihe um eine Tiefenbohrung zu genau diesem einen Werkzeug. Es richtet sich an dieselbe Zielgruppe: IT-Administrator:innen, Studierende, angehende Penetration Tester und alle, die eine solide, praxisnahe Grundlage in der Netzwerkerkundung suchen.

> **ILLEGAL – niemals ohne Auftrag**
>
> Wie in den anderen Büchern dieser Reihe gilt ohne Ausnahme: Jede Technik in diesem Handbuch ist **ausschließlich** für den Einsatz in einer eigenen, isolierten Testumgebung oder im Rahmen eines schriftlich autorisierten Auftrags bestimmt. Entscheidend sind Berechtigung, Zweck und Umfang des Tests – der Einsatz gegen fremde Systeme ohne entsprechende Berechtigung kann nach §§ 202a–202c StGB strafbar sein, unabhängig von der Absicht (Ethical-Hacking-Handbuch, Kapitel 1 und 5).

Für wen ist dieses Handbuch?

Du bist hier richtig, wenn du

- bereits das Ethical-Hacking-Handbuch kennst und Nmap als zentrales Werkzeug vertiefen möchtest,
- eigene Netzwerke systematisch inventarisieren oder auf offene Dienste prüfen willst,
- die Nmap Scripting Engine (NSE) für automatisierte Prüfungen nutzen möchtest,
- dich auf Zertifizierungen vorbereitest, in denen Nmap eine zentrale Rolle spielt (eJPT, OSCP, CompTIA Security+/PenTest+).

Aufbau

Kapitel	Thema
1	Was ist Nmap? Einordnung, Geschichte, Abgrenzung zu hping3
2	Host-Discovery: Wer ist überhaupt erreichbar?
3	Installation und Laborumgebung
4	Scan-Typen: TCP-, UDP- und weitere Verfahren
5	Service-, Versions- und Betriebssystemerkennung
6	Timing und Performance
7	Die Nmap Scripting Engine (NSE)
8	Firewall- und IDS-Evasion – ausschließlich im eigenen Labor
9	Ausgabeformate und Automatisierung
10	Praxisteil: Laboraufbau und geführte Übungen
11	Troubleshooting, Spickzettel, Glossar und Ressourcen

> **TIPP – Profi-Trick**
>
> Du kennst bereits das hping3-Handbuch dieser Reihe? Dann wird dir in Kapitel 2 und 4 auffallen, wie viel von dem Feldwissen (TCP-Flags, TTL, Fenstergröße) direkt wiederverwendbar ist – Nmap baut auf denselben Grundlagen auf, verpackt sie nur in robustere, automatisierte Abläufe.

Kapitel 1: Was ist Nmap?

Lernziele

- Du kannst Nmap von hping3 funktional abgrenzen.
- Du kennst die Geschichte und die wichtigsten Einsatzgebiete von Nmap.
- Du verstehst den grundsätzlichen Aufbau eines Nmap-Scans.

1.1 Einordnung: automatisierte Erkundung statt manueller Paketbau

Während `hping3` dir erlaubt, jedes Feld eines einzelnen Pakets selbst zu bestimmen (hping3-Handbuch, Kapitel 2), automatisiert Nmap ganze **Scan-Abläufe**: Es sendet systematisch Pakete an viele Ports und oft viele Hosts gleichzeitig, wertet die Antworten aus und fasst sie in einem übersichtlichen Bericht zusammen.

Werkzeug	Grundidee	Typischer Einsatz
`hping3`	manuell konstruierte Einzelpakete	gezielte Tiefenanalyse, Firewall-Experimente
`Nmap`	automatisierte Scan-Logik mit vielen Zielen	Netzwerk- und Diensterkennung in der Fläche, Inventarisierung

Gut zu wissen

Nmap wurde 1997 von Gordon Lyon (bekannt unter dem Pseudonym „Fyodor“) veröffentlicht und wird seither kontinuierlich weiterentwickelt. Mit der **Nmap Scripting Engine (NSE)**, eingeführt 2009, wurde aus dem reinen Portscanner ein vollständiges Rahmenwerk für automatisierte Netzwerk- und Schwachstellenprüfungen (Kapitel 7).

1.2 Typische Einsatzgebiete

- **Host-Discovery**: Welche Geräte sind in einem Netzwerk überhaupt aktiv? (Kapitel 2)
- **Portscanning**: Welche Dienste sind auf einem oder vielen Hosts erreichbar? (Kapitel 4)
- **Service- und Versionserkennung**: Welche Software mit welcher Version läuft hinter einem offenen Port? (Kapitel 5)
- **Betriebssystem-Fingerprinting**: Welches Betriebssystem läuft wahrscheinlich auf dem Zielsystem? (Kapitel 5)
- **Automatisierte Prüfungen über NSE**: von einfachen Bannerabfragen bis zu Schwachstellenchecks (Kapitel 7)
- **Netzwerkinventarisierung**: eine der häufigsten, unterschätzten Alltagsanwendungen – welche Geräte hängen überhaupt in meinem eigenen Netz?

1.3 Der grundsätzliche Scan-Ablauf

Beispiel 1.1: Was Nmap bei einem typischen Scan tut (Konzept)

```
1. Host-Discovery: Ist das Ziel überhaupt erreichbar? (Kapitel 2)
2. Portscanning: Welche Ports antworten, welche nicht? (Kapitel 4)
3. Service-/Versionserkennung: Was läuft auf den offenen Ports? (Kapitel 5)
4. Optional: Betriebssystem-Fingerprinting (Kapitel 5)
5. Optional: NSE-Skripte gegen erkannte Dienste ausführen (Kapitel 7)
6. Ausgabe als Bericht, wahlweise in mehreren Formaten (Kapitel 9)
```

Jeder dieser Schritte lässt sich einzeln ansteuern oder weglassen – ein einfacher Scan durchläuft oft nur Schritt 1 und 2, ein vollständiger Audit alle sechs.

1.4 Dual-Use wie jedes Erkundungswerkzeug

LEGAL – nur mit Auftrag/Einwilligung

Auch Nmap hat keine Funktion, die „nur für legale Zwecke“ reserviert ist. Netzwerkerkundung ist der erste Schritt sowohl eines beauftragten Pentests als auch eines unautorisierten Angriffs. Der einzige Unterschied ist – wie im gesamten Ethical-Hacking-Handbuch – die **Autorisierung**: ein schriftlicher Auftrag mit definiertem Scope oder ein System, das dir selbst gehört (Kapitel 3 und 10).

Zusammenfassung

Zusammenfassung

- Nmap automatisiert Scan-Abläufe über viele Ports und Hosts, während hping3 einzelne Pakete manuell konstruiert.
- Haupteinsatzgebiete sind Host-Discovery, Portscanning, Service-/OS-Erkennung, NSE-gestützte Prüfungen und Inventarisierung.
- Ein vollständiger Scan durchläuft bis zu sechs Schritte, die einzeln ansteuerbar sind.
- Wie jedes Erkundungswerkzeug entscheidet ausschließlich die Autorisierung über Legalität, nicht die Technik.

Kapitel 2: Host-Discovery

Lernziele

- Du verstehst, warum Host-Discovery dem eigentlichen Portscan vorausgeht.
- Du kennst die wichtigsten Discovery-Methoden und ihre Grenzen.
- Du kannst Discovery gezielt überspringen oder erzwingen.

2.1 Warum ein eigener Schritt vor dem Portscan?

Bevor Nmap Zeit in einen Portscan investiert, prüft es standardmäßig, ob ein Ziel überhaupt erreichbar ist – sonst würde ein Scan gegen tausend nicht existierende Adressen unnötig lange dauern. Dieser Schritt heißt **Host-Discovery** und lässt sich unabhängig vom eigentlichen Portscan steuern.

Beispiel 2.1: Nur Host-Discovery, kein Portscan

```
$ nmap -sn 192.168.56.0/24
Nmap scan report for 192.168.56.1
Host is up (0.00022s latency).
Nmap scan report for 192.168.56.102
Host is up (0.00031s latency).
Nmap done: 256 IP addresses (2 hosts up) scanned in 2.14 seconds
```

`-sn` (früher `-sP`) führt ausschließlich die Erreichbarkeitsprüfung durch, ganz ohne Portscan – ideal, um schnell eine Übersicht aktiver Geräte in einem Netzsegment zu bekommen.

2.2 Discovery-Methoden im Überblick

Methode	Option	Funktionsweise
ICMP Echo	`-PE`	klassischer Ping (ICMP Echo Request)
TCP SYN Ping	`-PS<ports>`	SYN-Pakete an angegebene Ports, wertet jede Antwort als „Host lebt“
TCP ACK Ping	`-PA<ports>`	ACK-Pakete, nützlich wenn SYN gefiltert wird
UDP Ping	`-PU<ports>`	UDP-Pakete, wertet ICMP-Fehlerantworten aus
ARP Ping	automatisch im lokalen Netz	zuverlässigste Methode im eigenen Subnetz, da ARP kaum gefiltert wird

Gut zu wissen

Im **lokalen Subnetz** nutzt Nmap automatisch ARP-Anfragen zur Discovery, weil sie auf Schicht 2 praktisch nie gefiltert werden und deutlich zuverlässiger sind als ICMP oder TCP. Das ist einer der Gründe, warum Scans im eigenen Testnetz oft deutlich zuverlässigere Ergebnisse liefern als Scans über mehrere Hops hinweg.

2.3 Discovery überspringen oder erzwingen

Beispiel 2.2: Discovery deaktivieren – jeden Host als "lebend" annehmen

```
$ nmap -Pn -p 80 192.168.56.102
```

`-Pn` überspringt die Erreichbarkeitsprüfung vollständig und scannt direkt. Das ist sinnvoll, wenn du bereits weißt, dass ein Host existiert, aber ICMP/ARP-Antworten durch eine Firewall gefiltert werden – ohne `-Pn` würde Nmap den Host sonst fälschlich als „nicht erreichbar“ einstufen und gar nicht erst scannen.

WARNUNG – Sicherheitsrisiko

Antwortet ein Ziel nicht auf die Discovery-Methoden, bedeutet das **nicht zwangsläufig**, dass es offline ist – es kann ebenso gut eine Firewall sein, die genau diese Anfragen filtert. Bei wichtigen Scans lohnt sich im Zweifel ein zusätzlicher Lauf mit `-Pn`, um diesen Unterschied nicht zu übersehen.

Zusammenfassung

Zusammenfassung

- Host-Discovery prüft vor dem eigentlichen Portscan, ob ein Ziel erreichbar ist, und spart dadurch Zeit.
- Im lokalen Subnetz nutzt Nmap automatisch die zuverlässige ARP-Methode.
- `-sn` führt nur die Discovery durch, `-Pn` überspringt sie vollständig.
- Keine Antwort auf Discovery-Anfragen ist mehrdeutig: offline oder gefiltert sind beide möglich.

Kapitel 3: Installation und Laborumgebung

Lernziele

- Du installierst Nmap unter Kali, Debian/Ubuntu und Fedora/Arch.
- Du nutzt dasselbe isolierte Testnetz wie in den anderen Büchern dieser Reihe.
- Du verifizierst, dass Nmap und Zielsystem im selben, vom Internet getrennten Netz liegen.

3.1 Installation

Beispiel 3.1: Nmap installieren

```
# Kali Linux: bereits vorinstalliert
$ nmap --version

# Debian/Ubuntu
$ sudo apt update && sudo apt install nmap

# Fedora
$ sudo dnf install nmap
# Arch Linux
$ sudo pacman -S nmap

# Windows (offiziell gepflegter Installer)
# Installer von der offiziellen Nmap-Projektseite laden und ausführen
```

Gut zu wissen

Nmap ist im Gegensatz zu hping3 auch unter Windows offiziell und gut gepflegt verfügbar, inklusive der grafischen Oberfläche **Zenmap**. Für dieses Buch reicht die Kommandozeilenversion unter Linux völlig aus; die Befehle sind plattformübergreifend identisch.

3.2 Root-Rechte: wann nötig, wann nicht

Beispiel 3.2: Einfacher TCP-Connect-Scan ohne besondere Rechte

```
$ nmap -sT -p 80 192.168.56.102
```

Beispiel 3.3: SYN-Scan mit den nötigen Rechten

```
$ sudo nmap -sS -p 80 192.168.56.102
```

Ein **TCP-Connect-Scan** (`-sT`) nutzt normale Betriebssystem-Sockets und braucht keine besonderen Rechte. Der **SYN-Scan** (`-sS`, Standard bei root-Rechten) erzeugt dagegen Rohpakete und braucht dieselben erweiterten Rechte wie hping3 – entweder `sudo` oder gezielt die Capability `cap_net_raw` (hping3-Handbuch, Kapitel 3.2).

3.3 Das isolierte Testnetz

Dieses Buch nutzt dasselbe Labor wie das Ethical-Hacking-Handbuch und das hping3-Handbuch: zwei virtuelle Maschinen in einem **Internal Network**, ohne Internetzugang.

Beispiel 3.4: Isolation vor jeder Übung verifizieren

```
$ ip route
# Keine Zeile mit "default via ..." -> kein bekannter Weg ins Internet
$ nmap -sn 192.168.56.0/24
# Nur deine eigenen Test-VMs sollten als "Host is up" erscheinen
```

LEGAL – nur mit Auftrag/Einwilligung

Scanne mit Nmap niemals einen Adressbereich, der größer ist als dein eigenes Testnetz, „nur um zu sehen, was da ist“. Auch ein reiner Host-Discovery-Scan (`-sn`) ist eine aktive Kontaktaufnahme und gehört ausschließlich in eine autorisierte oder vollständig eigene Umgebung.

Zusammenfassung

Zusammenfassung

- Nmap läuft nativ unter Linux, macOS und Windows; die Befehle sind weitgehend identisch.
- TCP-Connect-Scans brauchen keine besonderen Rechte, SYN-Scans und viele weitere Verfahren schon.
- Dasselbe isolierte Internal-Network-Labor wie in den anderen Büchern dieser Reihe reicht für alle Übungen aus.
- Auch ein reiner Discovery-Scan ist eine aktive Handlung und gehört ausschließlich ins eigene oder autorisierte Netz.

Kapitel 4: Scan-Typen

Lernziele

- Du kennst die wichtigsten TCP- und UDP-Scan-Verfahren und ihre Unterschiede.
- Du kannst Portbereiche und -listen gezielt angeben.
- Du verstehst, warum verschiedene Scan-Typen unterschiedlich zuverlässig und unterschiedlich auffällig sind.

4.1 Die wichtigsten TCP-Scan-Typen

Option	Name	Funktionsweise
`-sS`	SYN-Scan	sendet SYN, wertet SYN+ACK als offen, schließt die Verbindung nie vollständig ab (Standard mit root-Rechten)
`-sT`	Connect-Scan	nutzt den vollständigen Betriebssystem-Verbindungsaufbau, keine Root-Rechte nötig
`-sA`	ACK-Scan	erkennt nicht offen/geschlossen, sondern ob eine Firewall zustandsbehaftet filtert
`-sF` / `-sX` / `-sN`	FIN-, Xmas-, Null-Scan	ungewöhnliche Flag-Kombinationen, ursprünglich zur Firewall-Umgehung gedacht (Kapitel 8)

Beispiel 4.1: Ein klassischer SYN-Scan

```
$ sudo nmap -sS -p 1-1000 192.168.56.102
PORT     STATE SERVICE
21/tcp   open  ftp
22/tcp   open  ssh
80/tcp   open  http
3306/tcp open  mysql
```

Gut zu wissen

Der SYN-Scan gilt als „Halb-offener Scan“: Nmap sendet ein SYN, wertet die Antwort aus, schickt bei einem offenen Port aber nie das abschließende ACK, sondern direkt ein RST. Das ist schneller und hinterlässt in einfachen Logs weniger vollständige Verbindungen als ein Connect-Scan – es ist aber **kein Tarnmodus**: moderne Firewalls und IDS/IPS erkennen auch halb-offene Scans zuverlässig.

4.2 UDP-Scans

Beispiel 4.2: UDP-Scan (deutlich langsamer als TCP)

```
$ sudo nmap -sU -p 53,161 192.168.56.102
PORT    STATE         SERVICE
53/udp  open          domain
161/udp open|filtered snmp
```

Der Status `open|filtered` ist typisch für UDP: Bleibt eine Antwort aus, kann Nmap nicht sicher zwischen „offen, aber stumm“ und „durch eine Firewall gefiltert“ unterscheiden (dieselbe Mehrdeutigkeit wie bei hping3-UDP-Tests, hping3-Handbuch Kapitel 5.3). UDP-Scans sind außerdem spürbar langsamer, weil fehlende Antworten erst nach einem Timeout als mehrdeutig gewertet werden.

4.3 Ports gezielt auswählen

Beispiel 4.3: Portauswahl-Varianten

```
$ nmap -p 80 192.168.56.102             # ein einzelner Port
$ nmap -p 1-1000 192.168.56.102         # ein Bereich
$ nmap -p 22,80,443 192.168.56.102      # eine Liste
$ nmap -p- 192.168.56.102               # alle 65535 Ports
$ nmap -F 192.168.56.102                # nur die ~100 häufigsten Ports (schnell)
$ nmap --top-ports 20 192.168.56.102    # die 20 häufigsten Ports
```

TIPP – Profi-Trick

Für eine erste Übersicht ist `-F` (Fast-Modus, nur die häufigsten Ports) oft die bessere Wahl als `-p-`: Ein vollständiger Scan aller 65535 Ports dauert insbesondere bei UDP erheblich länger, liefert für die meisten Zwecke aber kaum zusätzliche Erkenntnisse.

Zusammenfassung

Zusammenfassung

- SYN-Scans sind der schnelle Standard mit root-Rechten, Connect-Scans die rechtefreie Alternative.
- ACK-, FIN-, Xmas- und Null-Scans dienen speziellen Zwecken wie Firewall-Analyse (Kapitel 8).
- UDP-Scans liefern mehrdeutigere Ergebnisse (`open|filtered`) und sind deutlich langsamer als TCP-Scans.
- Portauswahl lässt sich über Einzelport, Bereich, Liste, alle Ports oder vordefinierte Kurzlisten steuern.

Kapitel 5: Service-, Versions- und Betriebssystemerkennung

Lernziele

- Du nutzt Nmap zur Erkennung von Diensten und deren Versionen.
- Du setzt die Betriebssystem-Fingerprinting-Funktion sinnvoll und mit richtiger Erwartungshaltung ein.
- Du verstehst, worauf diese Erkennungen technisch beruhen.

5.1 Service- und Versionserkennung

Beispiel 5.1: Versionserkennung mit -sV

```
$ sudo nmap -sV -p 21,22,80 192.168.56.102
PORT   STATE SERVICE VERSION
21/tcp open  ftp     vsftpd 2.3.4
22/tcp open  ssh     OpenSSH 4.7p1 Debian 8ubuntu1
80/tcp open  http    Apache httpd 2.2.8 ((Ubuntu) DAV/2)
```

`-sV` geht über das reine Banner-Auslesen (hping3-Handbuch, Kapitel 4.2) hinaus: Nmap vergleicht beobachtete Antworten mit einer gepflegten Datenbank bekannter Dienst-Signaturen (`nmap-service-probes`) und sendet bei Bedarf zusätzliche, protokollspezifische Testanfragen, um die Version präziser einzugrenzen.

Gut zu wissen

Je nach Intensitätsstufe (`--version-intensity 0` bis `9`, Standard `7`) sendet Nmap mehr oder weniger zusätzliche Sondierungsanfragen. Eine niedrigere Intensität ist schneller und unauffälliger, erkennt aber unter Umständen weniger Details.

5.2 Betriebssystem-Fingerprinting

Beispiel 5.2: OS-Erkennung mit -O

```
$ sudo nmap -O 192.168.56.102
Running: Linux 3.X|4.X
OS CPE: cpe:/o:linux:linux_kernel:3 cpe:/o:linux:linux_kernel:4
OS details: Linux 3.2 - 4.9
Network Distance: 1 hop
```

Nmap kombiniert dafür deutlich mehr Merkmale als die TTL- und Fenstergrößen-Heuristik aus dem hping3-Handbuch (Kapitel 7.2–7.3): unter anderem das Verhalten bei besonders konstruierten Testpaketen, die Reihenfolge von TCP-Optionen und die Reaktion auf geschlossene Ports. Das Ergebnis ist deutlich zuverlässiger als eine reine TTL-Schätzung, bleibt aber eine **Wahrscheinlichkeitsaussage**.

WARNUNG – Sicherheitsrisiko

Betriebssystem-Fingerprinting funktioniert nur zuverlässig, wenn mindestens ein offener **und** ein geschlossener Port am Ziel gefunden werden. Firewalls, die alle nicht benötigten Ports stillschweigend verwerfen, können die Erkennung deutlich erschweren oder verfälschen.

5.3 Beides kombinieren

Beispiel 5.3: Service-, Versions- und OS-Erkennung in einem Lauf

```
$ sudo nmap -sV -O -p 1-1000 192.168.56.102
```

TIPP – Profi-Trick

Die Kombination `-A` (für „Aggressive“) aktiviert Versions- und OS-Erkennung zusammen mit Standard-NSE-Skripten (Kapitel 7) und einem Traceroute in einem einzigen Aufruf – bequem für eine erste, umfassende Übersicht, aber auch deutlich langsamer und auffälliger als ein gezielter Scan.

Zusammenfassung

Zusammenfassung

- `-sV` vergleicht Antworten mit einer Signaturdatenbank und sendet bei Bedarf gezielte Zusatzanfragen.
- `-O` kombiniert mehrere Verhaltensmerkmale zu einer deutlich zuverlässigeren, aber weiterhin wahrscheinlichkeitsbasierten OS-Schätzung.
- Für eine verlässliche OS-Erkennung braucht Nmap sowohl einen offenen als auch einen geschlossenen Port.
- `-A` bündelt Versionserkennung, OS-Erkennung, Standard-NSE-Skripte und Traceroute, ist dafür aber langsamer und auffälliger.

Kapitel 6: Timing und Performance

Lernziele

- Du kennst die sechs Timing-Templates und ihre Auswirkungen.
- Du kannst Scans gezielt beschleunigen oder bewusst verlangsamen.
- Du verstehst den Zielkonflikt zwischen Geschwindigkeit, Zuverlässigkeit und Auffälligkeit.

6.1 Die Timing-Templates

Beispiel 6.1: Timing-Template setzen

```
$ sudo nmap -T4 -p 1-1000 192.168.56.102
```

Template	Name	Charakteristik
`-T0`	Paranoid	extrem langsam, ein Paket alle 5 Minuten – zur IDS-Umgehung gedacht
`-T1`	Sneaky	sehr langsam
`-T2`	Polite	langsamer als Standard, schont Bandbreite und Zielsystem
`-T3`	Normal	Standardverhalten
`-T4`	Aggressive	schneller, für stabile, nahegelegene Netze (im eigenen Labor üblich)
`-T5`	Insane	sehr schnell, nimmt dafür eher Ungenauigkeiten in Kauf

TIPP – Profi-Trick

Für die Übungen in diesem Buch ist `-T4` im eigenen, stabilen Testnetz eine gute Wahl: deutlich schneller als der Standard, ohne die Zuverlässigkeit so stark zu beeinträchtigen wie `-T5`.

6.2 Feinere Kontrolle jenseits der Templates

Beispiel 6.2: Einzelne Timing-Parameter direkt setzen

```
$ sudo nmap --min-rate 100 --max-retries 2 -p 1-1000 192.168.56.102
```

Option	Wirkung
`--min-rate <n>`	sendet mindestens n Pakete pro Sekunde
`--max-rate <n>`	begrenzt die Rate nach oben
`--max-retries <n>`	wie oft ein unbeantwortetes Paket wiederholt wird
`--host-timeout <zeit>`	bricht die Prüfung eines einzelnen, sehr langsamen Hosts ab
`-T0` … `-T5`	bequeme Gesamtvorgabe, die mehrere Einzelparameter sinnvoll kombiniert

6.3 Der Zielkonflikt

Beispiel 6.3: Derselbe Scan mit unterschiedlichem Timing (Konzept)

```
-T2: sehr zuverlässig, aber langsam -> gut für instabile Netze oder sehr genaue Ergebnisse
-T4: guter Kompromiss im eigenen Labor
-T5: schnell, aber mit höherer Wahrscheinlichkeit für übersehene oder falsch gewertete Ports
```

> **Gut zu wissen**
>
> Schnelleres Timing erhöht die Wahrscheinlichkeit für Paketverlust und damit für scheinbar „gefilterte“ Ports, die in Wirklichkeit nur eine verlorene Antwort hatten (dieselbe Mehrdeutigkeit wie in Kapitel 4.2 beschrieben). Bei wichtigen Ergebnissen lohnt sich ein zweiter, langsamerer Lauf zur Bestätigung.

Zusammenfassung

> **Zusammenfassung**
>
> - Sechs Timing-Templates (`-T0` bis `-T5`) bündeln sinnvolle Voreinstellungen für unterschiedliche Geschwindigkeits-/Zuverlässigkeitsstufen.
> - Einzelparameter wie `--min-rate` und `--max-retries` erlauben feinere Kontrolle jenseits der Templates.
> - Schnelleres Scannen erhöht Geschwindigkeit, aber auch das Risiko von Paketverlust und damit scheinbar gefilterten Ergebnissen.
> - `-T4` ist im eigenen, stabilen Testnetz meist ein guter Kompromiss.

Kapitel 7: Die Nmap Scripting Engine (NSE)

Lernziele

- Du verstehst den Aufbau und Zweck der Nmap Scripting Engine.
- Du nutzt Skript-Kategorien gezielt statt pauschal „alle“ Skripte auszuführen.
- Du kannst ein einzelnes NSE-Skript gezielt gegen einen erkannten Dienst einsetzen.

7.1 Was ist NSE?

Die **Nmap Scripting Engine** erweitert Nmap um in der Sprache **Lua** geschriebene Skripte, die auf Basis der Scan-Ergebnisse weitere, oft protokollspezifische Prüfungen durchführen – von einfachen Bannerabfragen bis zu Schwachstellenchecks.

Beispiel 7.1: Installierte NSE-Skripte auflisten

```
$ ls /usr/share/nmap/scripts/ | wc -l
$ ls /usr/share/nmap/scripts/ | grep http | head -5
```

7.2 Skript-Kategorien

Kategorie	Zweck
`default` (`-sC`)	eine vorausgewählte Sammlung sicherer, informativer Standardskripte
`discovery`	zusätzliche Informationen über Dienste und Netzwerk sammeln
`safe`	Skripte, die als unkritisch für das Zielsystem gelten
`intrusive`	Skripte, die das Zielsystem stärker belasten oder beeinträchtigen können
`vuln`	Prüfung auf bekannte, konkrete Schwachstellen
`auth`	Prüfung auf Authentifizierungsschwächen (z. B. Standard-Zugangsdaten)

Beispiel 7.2: Standardskripte zusammen mit Versionserkennung

```
$ sudo nmap -sC -sV -p 21,22,80 192.168.56.102
```

WARNUNG – Sicherheitsrisiko

Skripte der Kategorie `intrusive` oder `vuln` können Dienste destabilisieren oder zu unerwarteten Reaktionen auf dem Zielsystem führen. Setze sie ausschließlich gegen Systeme ein, bei denen ein solches Risiko ausdrücklich akzeptiert wurde – im eigenen Testnetz oder mit entsprechender vertraglicher Freigabe.

7.3 Ein einzelnes Skript gezielt einsetzen

Beispiel 7.3: Ein bestimmtes Skript gegen einen erkannten Dienst

```
$ sudo nmap --script=ftp-anon -p 21 192.168.56.102
PORT   STATE SERVICE
21/tcp open  ftp
| ftp-anon: Anonymous FTP login allowed (FTP code 230)
```

Dieses Beispiel prüft gezielt, ob der FTP-Dienst anonyme Anmeldungen erlaubt – eine klassische, in Metasploitable 2 absichtlich vorhandene Fehlkonfiguration (Ethical-Hacking-Handbuch, Kapitel 4).

Beispiel 7.4: Mehrere Skripte einer Kategorie kombinieren

```
$ sudo nmap --script=safe -p 1-1000 192.168.56.102
$ sudo nmap --script="http-*" -p 80 192.168.56.102
```

7.4 Eigene Argumente an Skripte übergeben

Beispiel 7.5: Skript-Argumente setzen

```
$ sudo nmap --script=http-title --script-args http.useragent="Mozilla/5.0" -p 80 192.168.56.102
```

> **TIPP – Profi-Trick**
>
> `--script-help=<skriptname>` zeigt dir Beschreibung, Autor und verfügbare Argumente eines Skripts an, bevor du es einsetzt – gerade bei unbekannten Skripten eine sinnvolle Gewohnheit, bevor du sie gegen ein Testsystem richtest.

Zusammenfassung

> **Zusammenfassung**
>
> - NSE erweitert Nmap um Lua-Skripte für protokollspezifische Prüfungen bis hin zu Schwachstellenchecks.
> - Skript-Kategorien (`default`, `safe`, `intrusive`, `vuln`, `auth`, u. a.) steuern Umfang und Risiko gezielt.
> - Einzelne Skripte lassen sich präzise gegen einen erkannten Dienst richten, statt pauschal „alles“ auszuführen.
> - `intrusive`- und `vuln`-Skripte gehören ausschließlich in autorisierte oder eigene Testumgebungen.

Kapitel 8: Firewall- und IDS-Evasion

Lernziele

- Du kennst die wichtigsten Nmap-Techniken zur Firewall- und IDS-Umgehung.
- Du verstehst, wofür diese Techniken aus Verteidigersicht nützlich sind.
- Du kannst einschätzen, warum moderne Schutzsysteme viele dieser Techniken längst erkennen.

ILLEGAL – niemals ohne Auftrag

Dieses Kapitel behandelt Techniken, die Erkennungsmechanismen gezielt umgehen sollen. Jede Anwendung außerhalb eines vollständig isolierten, eigenen Testnetzes oder eines ausdrücklich autorisierten Auftrags kann strafbar sein (§§ 202a–202c StGB) und verstößt gegen die Grundregeln dieser Buchreihe. Der Wert dieses Kapitels liegt darin, die eigene Firewall- oder IDS-Konfiguration zu verstehen und zu härten, nicht darin, fremde Systeme zu täuschen.

8.1 Fragmentierung

Beispiel 8.1: Pakete in kleinere Fragmente aufteilen

```
$ sudo nmap -f -p 80 192.168.56.102
$ sudo nmap -ff -p 80 192.168.56.102   # noch kleinere Fragmente
```

`-f` teilt die ausgehenden Pakete in kleinere IP-Fragmente auf – historisch dazu gedacht, einfache paketfilternde Firewalls zu umgehen, die nur das erste Fragment prüfen (dasselbe Prinzip wie in hping3-Handbuch, Kapitel 6.2). Moderne Firewalls setzen Fragmente in aller Regel vor der Prüfung wieder zusammen, wodurch dieser Trick heute meist wirkungslos ist – als Lerninhalt und zum Testen der eigenen Firewall-Konfiguration bleibt er aber wertvoll.

8.2 Decoys: Scheinbar viele gleichzeitige Scanner

Beispiel 8.2: Scan mit Decoy-Adressen tarnen

```
$ sudo nmap -D 192.168.56.50,192.168.56.51,ME -p 80 192.168.56.102
```

`-D` lässt den Scan so aussehen, als käme er gleichzeitig von mehreren Adressen (den „Decoys“) und der eigenen – `ME` markiert dabei die Position der echten Quelladresse in der Liste. Das Ziel „sieht“ mehrere scheinbare Scanner und kann die tatsächliche Quelle schwerer herausfiltern.

WARNUNG – Sicherheitsrisiko

Decoy-Adressen sollten **existierende, erreichbare Hosts** sein, da sonst leicht erkennbare, unrealistische Verkehrsmuster entstehen. In einer Laborumgebung bedeutet das: Verwende ausschließlich Adressen aus deinem eigenen Testnetz, niemals beliebige fremde Adressen – auch als Decoy ist das eine Form von Identitätsvortäuschung gegenüber einem dritten System.

8.3 Quellport-Manipulation

Beispiel 8.3: Einen vertrauenswürdig wirkenden Quellport setzen

```
$ sudo nmap -g 53 -p 22 192.168.56.102
```

Analog zum Quellport-Test mit hping3 (hping3-Handbuch, Kapitel 6.3) prüft dieser Ansatz, ob eine Firewall bestimmten Quellports (z. B. 53 für DNS-Antworten) pauschal vertraut.

8.4 Idle Scan mit Nmap

Beispiel 8.4: Idle Scan, automatisiert

```
$ sudo nmap -sI 192.168.56.50 -p 1-1000 192.168.56.102
```

`-sI` automatisiert genau das Prinzip, das im hping3-Handbuch (Kapitel 7.1) Schritt für Schritt von Hand nachvollzogen wird: Ein dritter „Zombie“-Host mit vorhersagbarer IP-ID-Sequenz verschleiert die tatsächliche Quelladresse des Scans.

8.5 Warum Evasion heute begrenzt wirkt

> **Gut zu wissen**
>
> Die meisten dieser Techniken stammen aus einer Zeit, in der Firewalls und IDS/IPS deutlich einfacher gebaut waren. Moderne Systeme setzen Fragmente zusammen, erkennen ungewöhnliche Flag-Kombinationen, werten Decoy-Muster statistisch aus und protokollieren Quellport-Anomalien. Der eigentliche Lernwert dieses Kapitels liegt darin, **die eigene Firewall-/IDS-Konfiguration gegen genau diese Techniken zu testen** – nicht darin, sich auf ihre Wirksamkeit gegen reale, moderne Systeme zu verlassen.

Zusammenfassung

> **Zusammenfassung**
>
> - Fragmentierung, Decoys, Quellport-Manipulation und Idle Scan sind die klassischen Nmap-Evasion-Techniken.
> - Alle vier haben defensive Gegenstücke: Sie zeigen, was eine Firewall oder ein IDS erkennen und abwehren können sollte.
> - Moderne Schutzsysteme erkennen die meisten dieser Techniken zuverlässig; ihr Wert liegt heute vor allem im Testen der eigenen Verteidigung.
> - Jede Anwendung gehört ausschließlich in ein eigenes, isoliertes Testnetz oder einen autorisierten Auftrag.

Kapitel 9: Ausgabeformate und Automatisierung

Lernziele

- Du kennst die wichtigsten Nmap-Ausgabeformate und ihre jeweiligen Einsatzzwecke.
- Du speicherst Scan-Ergebnisse so, dass sie später weiterverarbeitet werden können.
- Du kombinierst Nmap-Ausgaben mit einfachen Shell-Werkzeugen.

9.1 Die vier Ausgabeformate

Option	Format	Einsatzzweck
`-oN <datei>`	normal (wie die Bildschirmausgabe)	zum Lesen durch Menschen
`-oX <datei>`	XML	strukturierte Weiterverarbeitung, Grundlage für viele Auswertungswerkzeuge
`-oG <datei>`	„grepable“ (veraltet, aber weit verbreitet)	schnelle Auswertung mit `grep`/`awk`
`-oA <basisname>`	alle drei Formate gleichzeitig	Standard für Dokumentation und Bericht (Ethical-Hacking-Handbuch, Kapitel 8)

Beispiel 9.1: Scan mit allen drei Formaten gleichzeitig speichern

```
$ sudo nmap -sV -p 1-1000 -oA scan_192-168-56-102 192.168.56.102
$ ls scan_192-168-56-102.*
scan_192-168-56-102.nmap  scan_192-168-56-102.xml  scan_192-168-56-102.gnmap
```

TIPP – Profi-Trick

`-oA` direkt bei jedem ernsthaften Scan mitlaufen zu lassen, kostet nichts und erspart dir später, einen Scan noch einmal wiederholen zu müssen, nur weil du ein anderes Format brauchst.

9.2 Ergebnisse mit Shell-Werkzeugen auswerten

Beispiel 9.2: Offene Ports aus der grepable-Ausgabe filtern

```
$ grep "open" scan_192-168-56-102.gnmap
$ grep -oP '\d+/open' scan_192-168-56-102.gnmap | sort -u
```

Beispiel 9.3: Nur Hosts mit offenem Port 3306 (MySQL) auflisten

```
$ grep "3306/open" scan_192-168-56-102.gnmap | cut -d' ' -f2
```

9.3 XML als Grundlage für weitere Werkzeuge

Gut zu wissen

Das XML-Format (`-oX`) ist die Grundlage für zahlreiche grafische Auswertungswerkzeuge und eigene Auswertungsskripte, weil es im Gegensatz zum grepable-Format eine vollständige, strukturierte Repräsentation aller Scan-Details enthält. Für eigene Python-Skripte eignet sich die Bibliothek `python-nmap` oder eine direkte XML-Verarbeitung mit der Standardbibliothek.

Beispiel 9.4: XML-Ausgabe mit xmllint grob lesbar formatieren

```
$ xmllint --format scan_192-168-56-102.xml | less
```

Zusammenfassung

Zusammenfassung

- `-oN`, `-oX`, `-oG` und `-oA` decken die wichtigsten Ausgabebedürfnisse ab; `-oA` ist für ernsthafte Scans meist die richtige Wahl.
- Die grepable-Ausgabe eignet sich für schnelle Shell-Auswertungen, ist aber veraltet und weniger vollständig als XML.
- XML ist die strukturierte Grundlage für eigene Skripte und viele Auswertungswerkzeuge.

Kapitel 10: Praxisteil – Laboraufbau und geführte Übungen

Lernziele

- Du baust das für dieses Buch nötige Labor auf.
- Du arbeitest acht Übungen durch, die alle vorherigen Kapitel praktisch verbinden.
- Du weißt, dass abweichende Ergebnisse nicht automatisch ein Fehler sind, sondern oft an Netzwerkkonfiguration, Timing oder Zielsystem liegen.

10.1 Laboraufbau

Dieses Buch nutzt dasselbe Labor-Grundgerüst wie das Ethical-Hacking-Handbuch und das hping3-Handbuch: Kali-VM und Metasploitable-2-VM im selben Internal Network, ohne Internetzugang (Kapitel 3).

10.2 Checkliste: sichere Laborumgebung

Checkliste 10.1: Vor jeder Übung

```
[ ] Netzwerkadapter aller beteiligten VMs steht auf "Internes Netzwerk"
[ ] `ip route` zeigt keine Standardroute ins Internet
[ ] Zielsystem ist eine VM, die dir selbst gehört, kein reales/fremdes System
[ ] Für Übung 7/8: eine dritte Test-VM mit aktivierbarer Firewall bereit
```

10.3 Übungen

Jede Übung folgt demselben Schema: Ziel, Voraussetzungen, Durchführung, erwartetes Ergebnis, Auswertung und Fehlerbehebung.

Übung 1: Host-Discovery im eigenen Testnetz

Ziel: Den Unterschied zwischen Discovery und Portscan praktisch erleben. **Voraussetzungen:** Kali-VM mit Nmap, mindestens zwei weitere VMs im selben Internal Network. **Durchführung:** Führe `nmap -sn 192.168.56.0/24` (Beispiel 2.1) gegen dein Testnetz aus. **Erwartetes Ergebnis:** Alle aktiven VMs erscheinen als „Host is up“, ohne dass ein einziger Port geprüft wurde. **Auswertung:** Vergleiche die Liste mit den VMs, die du tatsächlich gestartet hast – es sollten keine zusätzlichen, dir unbekannten Hosts auftauchen. **Fehlerbehebung:** Fehlt eine VM in der Liste, prüfe deren Netzwerkadapter-Einstellung und ob sie im selben Internal Network wie die Kali-VM hängt.

Übung 2: SYN-Scan gegen bekannte Ports

Ziel: Den Standard-Scan-Typ einsetzen und mit dem hping3-Einzeltest vergleichen. **Voraussetzungen:** Metasploitable-2-VM als Ziel. **Durchführung:** Führe `sudo nmap -sS -p 1-1000 192.168.56.102` aus (Beispiel 4.1). Prüfe danach mit hping3 (hping3-Handbuch, Beispiel 5.1) manuell zwei der gefundenen Ports gegen. **Erwartetes Ergebnis:** Beide Werkzeuge kommen zum selben Befund für dieselben Ports. **Auswertung:** Stimmen beide überein, bestätigt das sowohl den Nmap-Befund als auch dein Verständnis der hping3-Flags. **Fehlerbehebung:** Bei Abweichungen: Wiederhole den Nmap-Scan mit `-T2` statt `-T4` – ein zu aggressives Timing kann einzelne Antworten verlieren (Kapitel 6.3).

Übung 3: Service- und Versionserkennung

Ziel: Erkannte Portdienste mit konkreten Versionsangaben anreichern. **Voraussetzungen:** wie Übung 2. **Durchführung:** Führe `sudo nmap -sV -p 21,22,80,3306 192.168.56.102` aus (Beispiel 5.1). **Erwartetes Ergebnis:** Für jeden offenen Port erscheint eine Dienstbezeichnung mit Versionsangabe (z. B. „vsftpd 2.3.4“). **Auswertung:** Vergleiche die gefundene FTP-Version mit dem Wissen aus dem Ethical-Hacking-Handbuch (Kapitel 5.3) zur dort beschriebenen, historisch bekannten Backdoor. **Fehlerbehebung:** Erscheinen nur Portnummern ohne Versionsangabe, erhöhe `--version-intensity` oder prüfe, ob der Dienst tatsächlich antwortet (z. B. mit `nc`).

Übung 4: Betriebssystem-Fingerprinting

Ziel: Die OS-Erkennung einsetzen und mit der TTL-Schätzung aus dem hping3-Handbuch vergleichen. **Voraussetzungen:** wie Übung 2. **Durchführung:** Führe `sudo nmap -O 192.168.56.102` aus (Beispiel 5.2). **Erwartetes Ergebnis:** Eine Einschätzung in Richtung „Linux“, mit Versionsbereich und Confidence-Angabe. **Auswertung:** Vergleiche das Ergebnis mit deiner TTL-Schätzung aus dem hping3-Handbuch (Übung 3 dort) – Nmaps Einschätzung sollte präziser, aber in dieselbe Richtung weisen. **Fehlerbehebung:** Liefert Nmap keine verlässliche Schätzung, prüfe, ob sowohl ein offener als auch ein geschlossener Port gefunden wurden (Kapitel 5.2) – ohne beide ist die Erkennung unzuverlässig.

Übung 5: Ein gezieltes NSE-Skript einsetzen

Ziel: Ein einzelnes NSE-Skript bewusst statt einer ganzen Kategorie einsetzen. **Voraussetzungen:** offener FTP-Port auf der Ziel-VM (aus Übung 3 bekannt). **Durchführung:** Führe `sudo nmap --script=ftp-anon -p 21 192.168.56.102` aus (Beispiel 7.3). **Erwartetes Ergebnis:** Eine Meldung, ob anonyme FTP-Anmeldung erlaubt ist. **Auswertung:** Notiere, ob der Fund mit dem übereinstimmt, was du über die Ziel-VM bereits weißt. **Fehlerbehebung:** Meldet das Skript einen Fehler, prüfe mit `--script-help=ftp-anon`, welche Voraussetzungen es erwartet.

Übung 6: Firewall-Regel mit Fragmentierung testen

Ziel: Prüfen, ob erzwungene Fragmentierung eine einfache Firewall-Regel beeinflusst. **Voraussetzungen:** eine dritte Test-VM mit aktivierbarer `iptables`/`nftables`-Regel (wie hping3-Handbuch, Übung 4). **Durchführung:** Aktiviere dieselbe einfache Regel wie im hping3-Handbuch. Teste danach mit `sudo nmap -sS -p 80 <ziel>` und mit `sudo nmap -f -sS -p 80 <ziel>` (Beispiel 8.1). **Erwartetes Ergebnis:** Mit einer modernen, korrekt konfigurierten Regel ändert sich das Ergebnis zwischen beiden Läufen nicht. **Auswertung:** Kein Unterschied ist das „gute“ Ergebnis; ein Unterschied wäre ein Hinweis auf eine Lücke in der Fragment-Behandlung der Regel. **Fehlerbehebung:** Beobachtest du einen Unterschied, vergleiche mit dem identischen Test aus dem hping3-Handbuch (Übung 5 dort) – beide Werkzeuge sollten zum selben Befund kommen.

Übung 7: Timing-Templates vergleichen

Ziel: Den Zielkonflikt zwischen Geschwindigkeit und Zuverlässigkeit selbst messen. **Voraussetzungen:** wie Übung 2. **Durchführung:** Führe denselben Scan (`-p 1-1000`) dreimal mit `-T2`, `-T4` und `-T5` aus und stoppe jeweils die Laufzeit mit `time`. **Erwartetes Ergebnis:** Die Laufzeit sinkt deutlich von `-T2` über `-T4` zu `-T5`. **Auswertung:** Vergleiche auch die gefundenen offenen Ports zwischen den drei Läufen – im stabilen Testnetz sollten sie meist identisch sein. **Fehlerbehebung:** Findest du bei `-T5` weniger Ports als bei `-T2`, ist das ein praktischer Beleg für den in Kapitel 6.3 beschriebenen Zielkonflikt, kein Fehler deinerseits.

Übung 8: Ergebnisse speichern und auswerten

Ziel: Einen vollständigen Scan dokumentieren und die Ausgabe weiterverarbeiten. **Voraussetzungen:** wie Übung 2. **Durchführung:** Führe `sudo nmap -sV -p 1-1000 -oA uebung8` aus (Beispiel 9.1). Filtere danach mit `grep` alle offenen Ports aus der `.gnmap`-Datei (Beispiel 9.2). **Erwartetes Ergebnis:** Drei Dateien (`.nmap`, `.xml`, `.gnmap`) sowie eine gefilterte Liste offener Ports. **Auswertung:** Vergleiche die gefilterte Liste mit der Bildschirmausgabe aus Übung 2 – sie sollten übereinstimmen. **Fehlerbehebung:** Liefert `grep` keine Treffer, prüfe mit `cat uebung8.gnmap`, ob die Datei tatsächlich wie erwartet befüllt wurde.

Zusammenfassung

Zusammenfassung

- Dasselbe Internal-Network-Labor wie in den anderen Büchern dieser Reihe reicht für alle acht Übungen aus.
- Die Übungen verbinden Discovery, Scanning, Service-/OS-Erkennung, NSE, Firewall-Tests, Timing und Dokumentation.
- Mehrere Übungen verweisen bewusst auf das hping3-Handbuch, um dieselbe Fragestellung mit beiden Werkzeugen zu vergleichen.

Kapitel 11: Troubleshooting, Spickzettel, Glossar und Ressourcen

Lernziele

- Du kannst die häufigsten Nmap-Fehlermeldungen einordnen und beheben.
- Du hast einen kompakten Spickzettel für den Alltag zur Hand.
- Du kennst weiterführende Ressourcen für die Vertiefung.

11.1 Häufige Probleme

Meldung / Symptom	Wahrscheinliche Ursache	Lösung
„You requested a scan type which requires root privileges“	SYN-Scan oder OS-Erkennung ohne ausreichende Rechte	`sudo` nutzen oder `cap_net_raw`-Capability setzen
Alle Ports erscheinen als `filtered`	Firewall blockiert, oder Discovery schlägt fehl	`-Pn` ergänzen, um Discovery zu überspringen (Kapitel 2.3)
Scan dauert ungewöhnlich lange	UDP-Scan, zu großer Portbereich, oder zu langsames Timing	Portauswahl eingrenzen (`-F`/`--top-ports`), `-T4` statt `-T2` prüfen
Versions-/OS-Erkennung liefert nichts Brauchbares	zu wenige offene **und** geschlossene Ports gefunden	größeren Portbereich scannen, `--version-intensity` erhöhen
NSE-Skript meldet einen Fehler	fehlende Voraussetzungen oder falsche Argumente	`--script-help=<skriptname>` prüfen
Ergebnisse weichen zwischen zwei Läufen ab	Paketverlust, insbesondere bei aggressivem Timing	Mit `-T2` oder erhöhtem `--max-retries` wiederholen

11.2 Spickzettel

Beispiel 11.1: Die wichtigsten Nmap-Befehle auf einen Blick

```
# Nur Host-Discovery, kein Portscan
nmap -sn <netz>/<maske>

# Klassischer SYN-Scan mit Versionserkennung
sudo nmap -sS -sV -p 1-1000 <ziel>

# Schnelle Übersicht der häufigsten Ports
nmap -F <ziel>
# Service-, Versions- und OS-Erkennung kombiniert
sudo nmap -sV -O <ziel>

# Standard-NSE-Skripte zusammen mit Versionserkennung
sudo nmap -sC -sV <ziel>

# Ein einzelnes NSE-Skript gezielt einsetzen
sudo nmap --script=<skriptname> -p <port> <ziel>

# Vollständiger, dokumentierter Lauf
sudo nmap -sV -O -sC -p- -oA bericht <ziel>
```

11.3 Glossar

- **ARP Ping** – Discovery-Methode im lokalen Subnetz über die Adressauflösung auf Schicht 2, selten gefiltert.
- **Decoy** – eine zusätzliche, scheinbare Quelladresse, die einen Scan zwischen mehreren Adressen verschleiert.
- **Host-Discovery** – der Schritt vor dem eigentlichen Portscan, der prüft, ob ein Ziel erreichbar ist.
- **Lua** – die Skriptsprache, in der NSE-Skripte geschrieben sind.
- **NSE (Nmap Scripting Engine)** – das Rahmenwerk für automatisierte, protokollspezifische Prüfungen auf Basis von Scan-Ergebnissen.
- **OS-Fingerprinting** – die Schätzung des Zielbetriebssystems anhand mehrerer kombinierter Verhaltensmerkmale.
- **SYN-Scan** – ein halb-offener TCP-Scan, der die Verbindung nie vollständig abschließt.
- **Timing-Template** – eine von sechs Voreinstellungen (`-T0`–`-T5`), die Geschwindigkeit gegen Zuverlässigkeit abwägen.
- **Versionserkennung** – der Abgleich beobachteter Dienstantworten mit einer Signaturdatenbank zur Versionsbestimmung.

11.4 Weiterführende Ressourcen

Ressource	Inhalt
Offizielles Nmap-Buch „Nmap Network Scanning“	umfassendes, vom Autor selbst verfasstes Referenzwerk, online frei verfügbar
Offizielle Nmap-Dokumentation (`man nmap`, nmap.org)	vollständige, verbindliche Optionsreferenz
hping3-Handbuch (dieselbe Buchreihe)	manuelle Paketerzeugung auf Feldebene als Ergänzung zu Nmaps automatisierten Abläufen
Ethical-Hacking-Handbuch (dieselbe Buchreihe)	Gesamtzusammenhang: Recon, Scanning, Exploitation, Reporting
TryHackMe / HackTheBox	legale Übungsplattformen zur Vertiefung von Scanning-Techniken
NSE-Skriptverzeichnis (`/usr/share/nmap/scripts/`)	lokale Referenz aller installierten Skripte mit Quelltext

TIPP – Profi-Trick

Die offizielle NSE-Dokumentation listet jedes Skript mit Beschreibung, Kategorie und Beispielausgabe. Bevor du ein unbekanntes Skript gegen ein Testsystem richtest, lohnt sich ein Blick hinein – auch `safe` eingestufte Skripte sind nicht automatisch für jeden Dienst sinnvoll.

Zusammenfassung

Zusammenfassung

- Die häufigsten Probleme betreffen fehlende Rechte, zu aggressive Firewalls oder unzureichende Portauswahl für Service-/OS-Erkennung.
- Der Spickzettel deckt die sieben häufigsten Befehlsmuster des gesamten Buches ab.

- Das offizielle Nmap-Buch, die Manpage, das hping3-Handbuch und das Ethical-Hacking-Handbuch sind die wichtigsten Anschlussstellen zum Weiterlernen.

Verzeichnis der Befehlsbeispiele

Beispiel 1.1: Was Nmap bei einem typischen Scan tut (Konzept) … Seite 8
Beispiel 2.1: Nur Host-Discovery, kein Portscan … Seite 9
Beispiel 2.2: Discovery deaktivieren – jeden Host als "lebend" annehmen … Seite 10
Beispiel 3.1: Nmap installieren … Seite 11
Beispiel 3.2: Einfacher TCP-Connect-Scan ohne besondere Rechte … Seite 11
Beispiel 3.3: SYN-Scan mit den nötigen Rechten … Seite 11
Beispiel 3.4: Isolation vor jeder Übung verifizieren … Seite 12
Beispiel 4.1: Ein klassischer SYN-Scan … Seite 13
Beispiel 4.2: UDP-Scan (deutlich langsamer als TCP) … Seite 13
Beispiel 4.3: Portauswahl-Varianten … Seite 14
Beispiel 5.1: Versionserkennung mit -sV … Seite 15
Beispiel 5.2: OS-Erkennung mit -O … Seite 15
Beispiel 5.3: Service-, Versions- und OS-Erkennung in einem Lauf … Seite 16
Beispiel 6.1: Timing-Template setzen … Seite 17
Beispiel 6.2: Einzelne Timing-Parameter direkt setzen … Seite 17
Beispiel 6.3: Derselbe Scan mit unterschiedlichem Timing (Konzept) … Seite 18
Beispiel 7.1: Installierte NSE-Skripte auflisten … Seite 19
Beispiel 7.2: Standardskripte zusammen mit Versionserkennung … Seite 19
Beispiel 7.3: Ein bestimmtes Skript gegen einen erkannten Dienst … Seite 20
Beispiel 7.4: Mehrere Skripte einer Kategorie kombinieren … Seite 20
Beispiel 7.5: Skript-Argumente setzen … Seite 20
Beispiel 8.1: Pakete in kleinere Fragmente aufteilen … Seite 21
Beispiel 8.2: Scan mit Decoy-Adressen tarnen … Seite 21
Beispiel 8.3: Einen vertrauenswürdig wirkenden Quellport setzen … Seite 22
Beispiel 8.4: Idle Scan, automatisiert … Seite 22
Beispiel 9.1: Scan mit allen drei Formaten gleichzeitig speichern … Seite 23
Beispiel 9.2: Offene Ports aus der grepable-Ausgabe filtern … Seite 23
Beispiel 9.3: Nur Hosts mit offenem Port 3306 (MySQL) auflisten … Seite 23
Beispiel 9.4: XML-Ausgabe mit xmllint grob lesbar formatieren … Seite 24
Checkliste 10.1: Vor jeder Übung … Seite 25
Beispiel 11.1: Die wichtigsten Nmap-Befehle auf einen Blick … Seite 28

Index

--script – 26
-O – 15, 26, 27, 29
-oA – 27
-sC – 11, 13, 14, 15, 26, 29
-sV – 15, 26, 27
ARP Ping – 29
Ausgabeformat – 23
Connect-Scan – 11
Decoy – 21, 29
Fragmentierung – 21, 26
grepable – 23
Host-Discovery – 7, 9, 25, 29
Idle Scan – 22
Lua – 19, 29
Nmap – 5, 7, 8, 9, 10, 11, 14, 15
NSE – 5, 7, 15, 17, 19, 20, 26, 27
OS-Fingerprinting – 29
Portauswahl – 14
Quellport – 22
SYN-Scan – 11, 13, 26, 29
Timing-Template – 17, 27, 29
UDP-Scan – 13, 14
Versionserkennung – 7, 15, 19, 26, 29
XML – 23, 24, 27

NMAP DAS PRAXISHANDBUCH

Vom einzelnen Host zur vollständigen Netzwerkkarte

Nmap automatisiert, was hping3 Paket für Paket von Hand erledigt: Host-Discovery, Portscans über ganze Netzbereiche, Service- und Betriebssystemerkennung und mit der Scripting Engine (NSE) sogar automatisierte Schwachstellenprüfungen. Dieses Handbuch führt von den Grundlagen bis zur Firewall-Evasion und zeigt durchgehend, wie Nmap und hping3 einander sinnvoll ergänzen.

DAS ERWARTET DICH

- Host-Discovery und die wichtigsten TCP-/UDP-Scan-Typen im Vergleich
- Service-, Versions- und Betriebssystemerkennung mit realistischer Fehlertoleranz
- Timing-Templates und der Zielkonflikt zwischen Tempo und Zuverlässigkeit
- Die Nmap Scripting Engine (NSE): Kategorien, gezielte Skripte, eigene Argumente
- Firewall-/IDS-Evasion, Ausgabeformate, 8 geführte Übungen, Spickzettel und Glossar

Für wen?

Leser:innen des Ethical-Hacking- und hping3-Handbuchs, die das zentrale Werkzeug der Netzwerkerkundung vertiefen möchten, sowie Studierende und Auszubildende. Ausschließlich für legale, autorisierte Tests und eigene Laborumgebungen.

32 Seiten · Stand Oktober 2026 · Ergänzung zum Ethical-Hacking- und hping3-Handbuch

Hinweis: Keine Rechts- oder Anlageberatung. Marken- und Produktnamen gehören ihren jeweiligen Inhabern.